

Algorithmique Avancée – Travail Pratique

Le Problème du Labyrinthe (*Rat in a Maze*)

1. Contexte du problème

Une souris se trouve à la case départ (en haut à gauche) d'un labyrinthe représenté par une grille carrée en deux dimensions de taille $N \times N$. Elle doit trouver un chemin pour atteindre la case d'arrivée (située tout en bas à droite) afin d'y récupérer un morceau de fromage.

Le labyrinthe est modélisé par une liste de listes (matrice) d'entiers :

* Les cases contenant un **0** représentent les couloirs **libres** où la souris peut circuler.

* Les cases contenant un **1** représentent les **murs** infranchissables.

2. Règles de déplacement

Pour simplifier l'exploration, la souris ne peut se déplacer que dans **deux directions** :

1. Vers le **Bas** (augmenter l'indice des lignes : $x + 1$)

2. Vers la **Droite** (augmenter l'indice des colonnes : $y + 1$)

Il est interdit de sortir des limites de la grille ou de marcher sur un mur.

3. Objectif

Vous devez écrire une fonction Python `resoudre_labyrinthe(lab)` utilisant le paradigme du **Backtracking** (exploration avec retour sur ses pas).

Spécifications de la fonction :

* **Entrée** : Une matrice `lab` de taille $N \times N$ contenant des `0` et des `1`.

* **Sortie** : * Si un chemin existe, la fonction doit renvoyer une **nouvelle matrice** de taille $N \times N$ appelée `solution`. Dans cette matrice, les cases empruntées par le chemin final doivent valoir `1`, et toutes les autres doivent valoir `0`.

* Si aucun chemin n'est possible, la fonction doit renvoyer `None`.

Exemple visuel :

Pour le labyrinthe suivant à gauche, votre fonction doit calculer et retourner la matrice de droite :

Labyrinthe d'entrée :

```
```python
```

```
lab = [
```

```
[0, 1, 0, 0],
```

```
[0, 0, 1, 0],
```

```
[1, 0, 0, 0],
```

```
[1, 1, 1, 0]
```

```
]
```

```
```
```

Matrice Solution attendue :

```
```python
```

```
La souris a avancé en (0,0) -> (1,0) -> (1,1) -> (2,1) ->
(2,2) -> (2,3) -> (3,3)
```

```
solution = [
```

```
[1, 0, 0, 0],
```

```
[1, 1, 0, 0],
```

```
[0, 1, 1, 1],
```

```
[0, 0, 0, 1]
```

---

]

---

\\

---

---

---

## ### 4. Contrainte d'implémentation (Barème)

---

La notation portera une attention cruciale à la gestion de la mémoire et à la propreté de l'algorithme :

---

> **Attention au nettoyage des impasses** : Si votre algorithme s'engage dans un couloir qui mène à un cul-de-sac, il doit obligatoirement faire marche arrière et **effacer sa trace** (remettre la case à 0 dans la matrice solution) avant d'explorer une autre direction. Une solution contenant des morceaux de chemins abandonnés sera considérée comme fausse.

---

---

---

Tests

```
def extraire_case_arrivee(solution):
 """Renvoie la valeur de la case d'arrivée en bas à droite."""
 if not solution or isinstance(solution, str):
 return 0
 return solution[-1][-1]

def extraire_case_depart(solution):
 """Renvoie la valeur de la case de départ en haut à gauche."""
 if not solution or isinstance(solution, str):
 return 0
 return solution[0][0]

Test 1 : Cas de base - Labyrinthe miniature 2x2 tout ouvert
lab_simple = [
 [0, 0],
```

```
[0, 0]
]
res_1 = resoudre_labyrinthe(lab_simple)
print("Test 1...", res_1)
assert extraire_case_arrivee(res_1) == 1, "Échec Test 1 : Un labyrinthe simple et ouvert doit être résolu."

Test 2 : Pas de solution - Arrivée bloquée par un mur
lab_bloque = [
 [0, 1],
 [0, 1]
]
res_2 = resoudre_labyrinthe(lab_bloque)
print("Test 2...", res_2)
assert res_2 is None or "Pas de solution" in str(res_2) or res_2 == False, "Échec Test 2 : Si l'arrivée est un mur, l'algorithme doit retourner un échec."

Test 3 : Départ bloqué immédiatement
lab_depart_bloque = [
 [1, 0],
 [0, 0]
]
res_3 = resoudre_labyrinthe(lab_depart_bloque)
print("Test 3...", res_3)
assert extraire_case_depart(res_3) == 0, "Échec Test 3 : Si la case de départ est un mur, impossible de commencer."

Test 4 : L'exemple classique du cours (avec un seul chemin sinueux)
lab_cours = [
 [0, 1, 0, 0],
 [0, 0, 1, 0],
 [1, 0, 0, 0],
 [1, 1, 1, 0]
]
res_4 = resoudre_labyrinthe(lab_cours)
print("Test 4...")
On vérifie un point clé du chemin obligatoire (la case (1, 1) et (2, 1))
assert res_4[1][1] == 1 and res_4[2][1] == 1 and res_4[3][3] == 1, "Échec Test 4 : Le chemin trouvé n'est pas le bon ou a sauté une étape clé."

Test 5 : Le piège du cul-de-sac (Force le Backtracking)
L'algorithme va vouloir aller tout droit (à droite), va s'enfoncer, bloquer, et devoir TOUT reculer
lab_piege = [
 [0, 0, 0, 1],
 [1, 1, 0, 1],
 [0, 0, 0, 0],
 [0, 1, 1, 0]
]
res_5 = resoudre_labyrinthe(lab_piege)
```

```
print("Test 5...")
Si le backtracking a mal nettoyé ses traces, la case (0, 3) ou (1, 2) pourrait
être restée à 1 indûment
assert res_5[0][1] == 1 and res_5[0][2] == 1, "Échec Test 5 : L'algorithme s'est
perdu dans le cul-de-sac ou n'a pas nettoyé ses pas."

Test 6 : Labyrinthe géant (Performance de l'exploration)
Une longue ligne droite qui serpente en zig-zag
taille = 30
lab_geant = [[0] * taille for _ in range(taille)]
On remplit de murs pour faire un unique serpent
for i in range(taille):
 for j in range(taille):
 if i % 2 == 1 and j < taille - 1:
 lab_geant[i][j] = 1
res_6 = resoudre_labyrinthe(lab_geant)
print("Test 6 (Test de performance)...")
assert extraire_case_arrivee(res_6) == 1, "Échec Test 6 : L'algorithme a planté
(Stack Overflow ?) ou est trop lent sur une grande grille."

Test 7 : Le piège du départ sans issue (Test de nettoyage absolu du path)
lab_impasse_depart = [
 [0, 1, 1],
 [1, 1, 1],
 [1, 1, 0] # L'arrivée est là, mais le départ est muré de tous les côtés
]

print("Test 7 (Test de l'impasse initiale)...")
res_7 = resoudre_labyrinthe(lab_impasse_depart)

On vérifie que la fonction renvoie bien None
assert res_7 is None, "Échec Test 7 : La fonction aurait dû renvoyer None car le
labyrinthe est impossible."

Test 8 : Le Grand Escalier avec fausses pistes (Compatible Bas/Droite
uniquement)
size = 25
lab_geant = [[0] * size for _ in range(size)]

1. On crée une structure en escalier (le vrai chemin suit la diagonale)
Mais on ouvre de grands boulevards "pièges" tout droit et vers le bas
for i in range(size):
 for j in range(size):
 # On mure les zones inutiles pour forcer un entonnoir diagonal
 if j > i + 1 or i > j + 1:
 lab_geant[i][j] = 1

2. LES PIÈGES : On ouvre des couloirs qui vont tout droit vers les bords...
mais qui finissent par des culs-de-sac tout au bout.
for i in range(0, size - 2, 3):
 # Fausse piste horizontale
 lab_geant[i][i+1] = 0
 if i + 2 < size:
```

```
 lab_geant[i][i+2] = 0
 if i + 3 < size:
 lab_geant[i][i+3] = 1 # Cul-de-sac !

print(f"Test 8 (Labyrinthe Géant {size}x{size} en Escalier)...")
res_8 = resoudre_labyrinthe(lab_geant)

Assertions pour valider le résultat
assert res_8 is not None, "Échec Test 8 : Le grand escalier a une solution valide en Bas/Droite !"
assert res_8[size-1][size-1] == 1, "Échec Test 8 : L'arrivée en bas à droite n'est pas atteinte."
On vérifie qu'une des fausses pistes (ex: ligne 3, colonne 6) a bien été nettoyée (vaut 0)
assert res_8[3][5] == 0, "Échec Test 8 : Le backtracking n'a pas nettoyé les fausses pistes de l'escalier !"

print("\nTous les tests du Labyrinthe sont passés avec succès !")
````
```